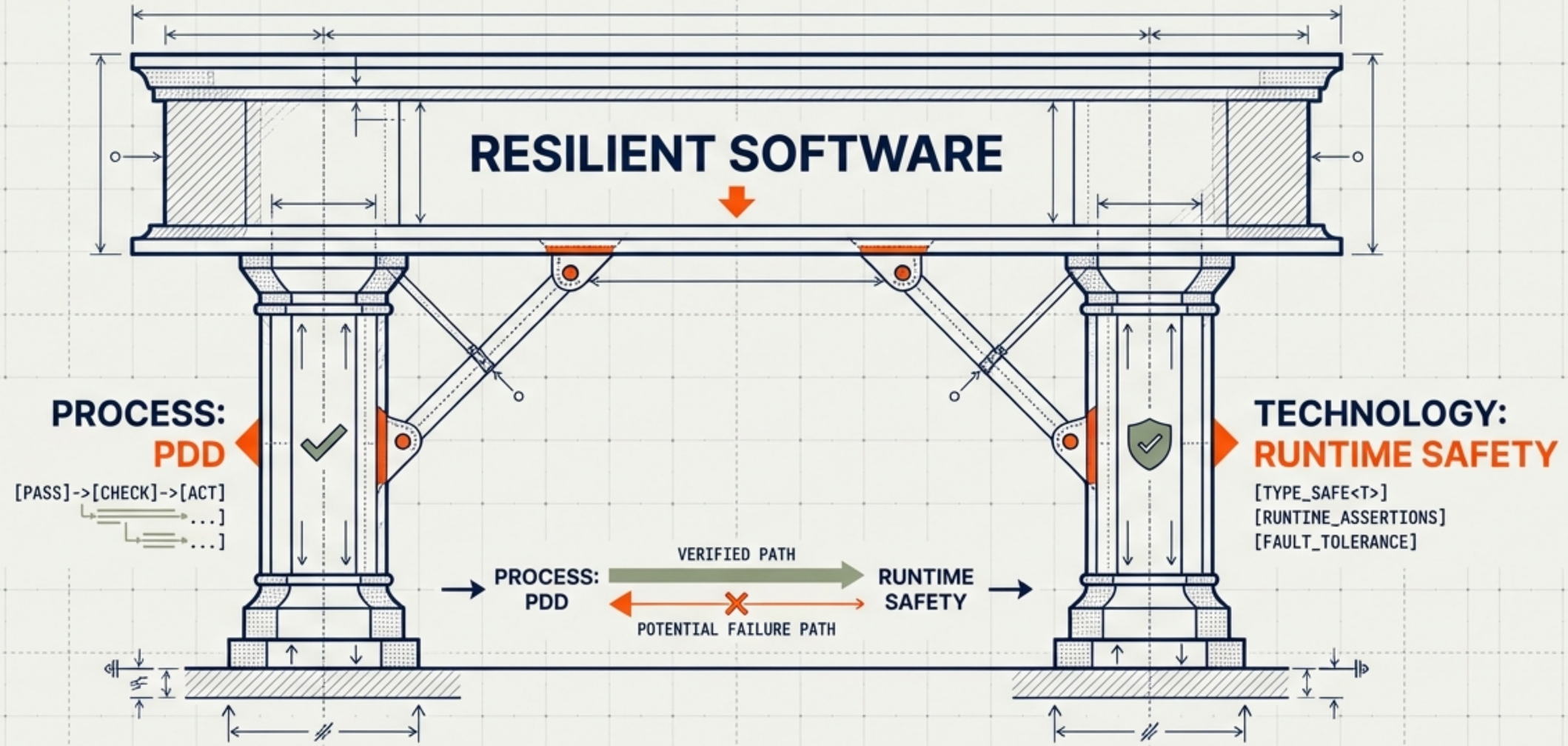
Engineering Resilient Software Beyond Static Checks

Bridging the gap between compile-time promises and runtime reality using Pass-Driven Development (PDD) and Type_Safe primitives.



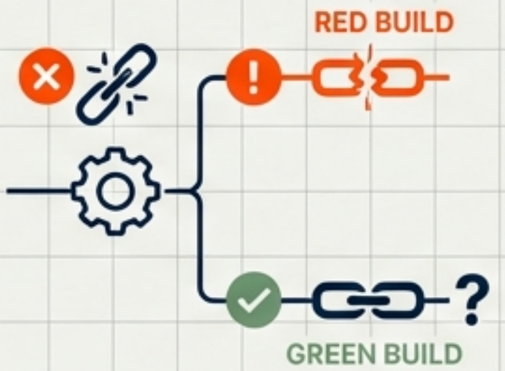
BASED ON THE RESEARCH AND ENGINEERING PRACTICES OF DINIS CRUZ AND OSBOT-UTILS.

THE REALITY OF DYNAMIC CODEBASES

THE CI CONFLICT (PROCESS)

Traditional Test-Driven Development (TDD) often creates friction in fast-moving projects.

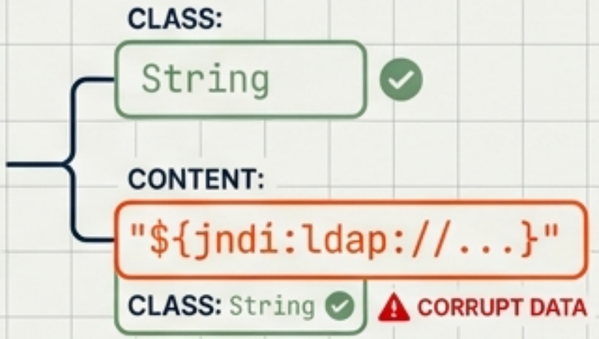
Writing failing tests for known bugs breaks the Continuous Integration (CI) pipeline. Teams hesitate to add tests to keep the build green, leaving edge cases untested.



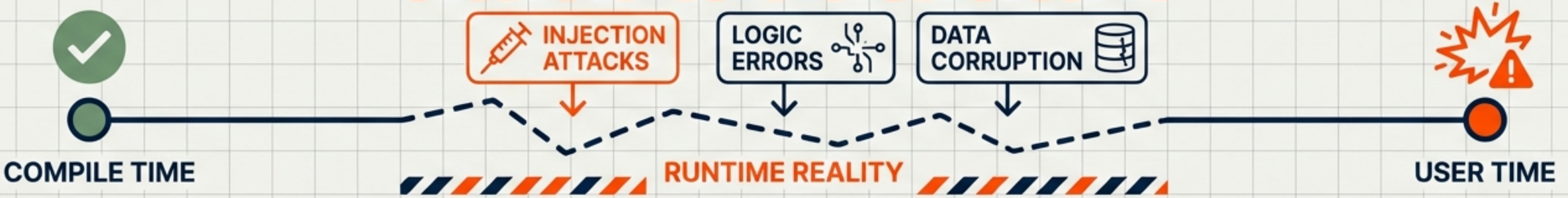
THE SEMANTIC BLINDSPOT (TECHNOLOGY)

Static type checkers (like mypy) validate classes, not content. To a compiler, a malicious payload is just a valid string.

Log4Shell (2021) and Equifax (2017) were caused by valid types carrying corrupt data.



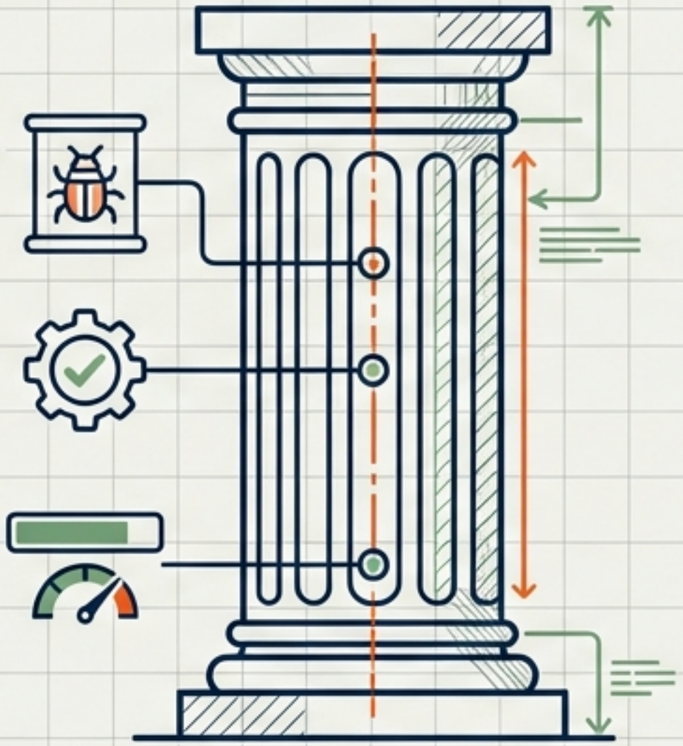
THE SAFETY GAP



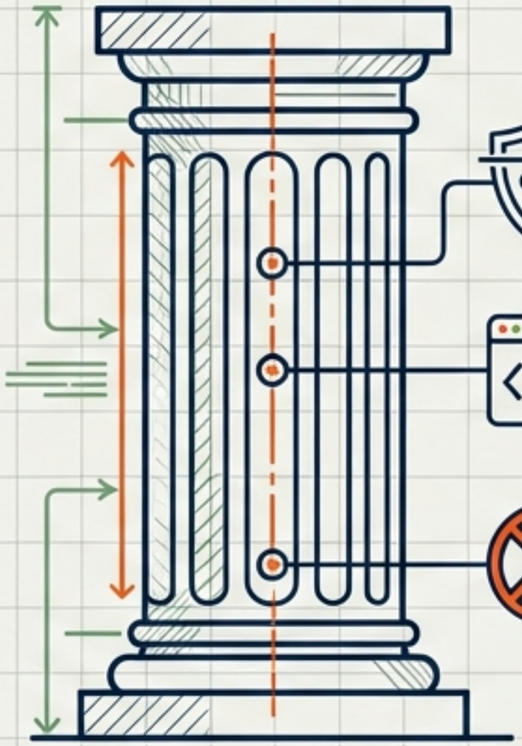
THE TWIN PILLARS OF SOFTWARE RESILIENCE.

PASS-DRIVEN DEVELOPMENT (PROCESS)

- Captures bugs without breaking builds
- "No bug left behind" via passing tests
- Outcome: Green builds + High coverage



RUNTIME TYPE SAFETY (TECHNOLOGY)



- Enforces correctness during execution
- OSBot-Utils Type_Safe primitives
- Outcome: Impossible states are unrepresentable

These are not opposing forces; they are complementary layers.
PDD documents behaviour; Type Safety validates data at the source.

FLIPPING THE SCRIPT ON TEST-DRIVEN DEVELOPMENT

THE PROBLEM

THE PROBLEM

Standard TDD:
Write failing test -> Break Build -> Fix

THE PDD SOLUTION

THE PDD RULE: KEEP THE BUILD GREEN

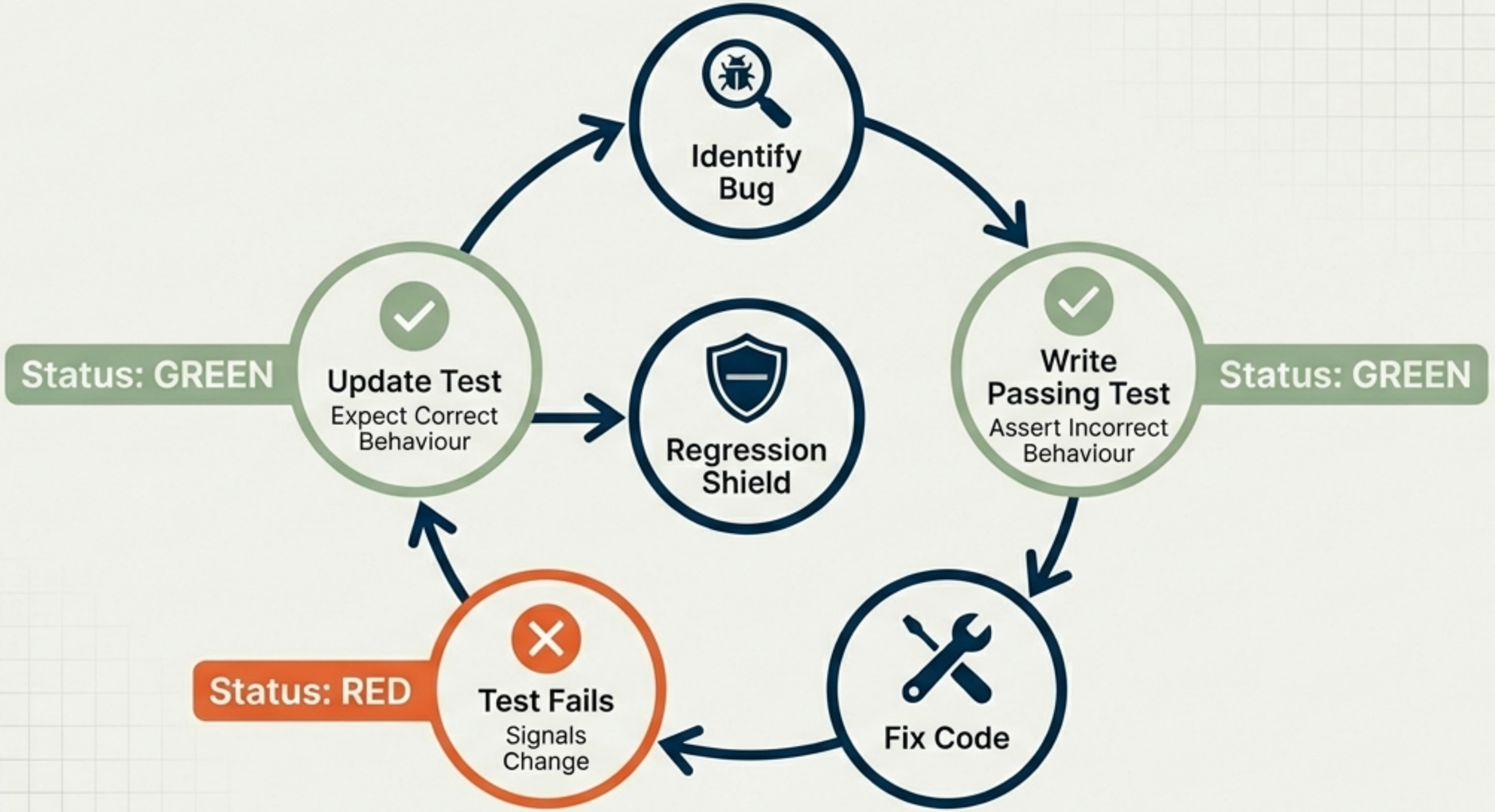
Instead of writing a test that asserts the correct result (breaking CI), write a test that asserts the current buggy behaviour. This transforms the test from a blocker into a living record.

```
def test_bug_dict_dont_support_type_checks(self):
    # Asserts the LIMITATION, not the fix
    assert my_dict.supports_types() is False
```

← Asserts the bug exists. Test passes. CI stays green.

PDD documents behaviour; Type Safety validates data at the source.

THE EVOLUTIONARY LIFECYCLE OF A PDD TEST.



The test flips from a Bug Existence record to a Regression Shield.

Strategic Advantages of the Pass-Driven Workflow.

Non-Disruptive CI

Bugs are catalogued immediately without halting momentum. Fixes occur on separate branches while the main pipeline stays green.

Full Traceability

Every test tells a story. Tests link directly to user requirements or bug reports.

"The test suite tells a story of the code's evolution."

Natural Coverage Growth

Coverage grows as a byproduct of addressing issues, not arbitrary targets. No bug left behind.

The Limits of Type Hints and the ‘String Problem’.

To a static checker, a malicious payload is just a valid string.

Compare and Contrast

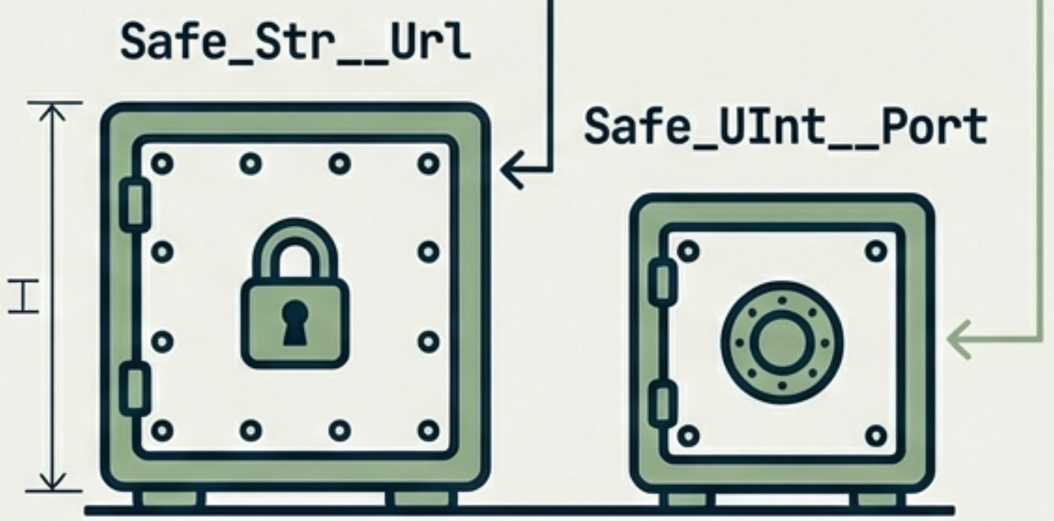
Standard Python (Unsafe)

```
def connect(host: str, port: int):
    ...
```

Vulnerable to injection and out-of-bounds errors.

OSBot-Utils Type_Safe

```
def connect(host: Safe_Str__Url, port: Safe_UInt__Port):
    ...
```



Enforced constraints. Primitives are banned.

Architecture of Runtime Enforcement.

Validation happens on assignment, preventing impossible states.

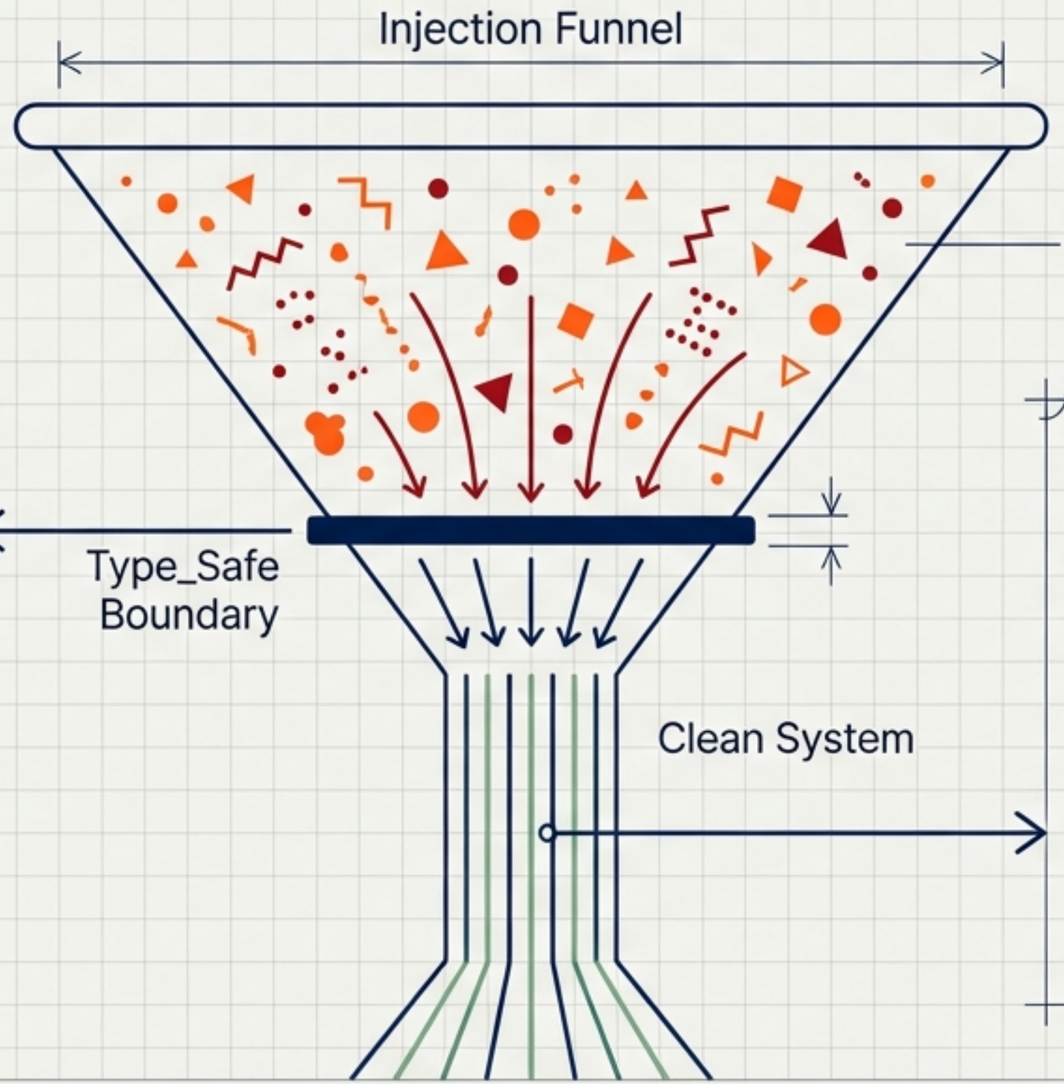
```
class ServerConfig(Type_Safe):
    port: Safe_UInt__Port                # Enforces 0-65535
    cpu_limit: Safe_UInt__Percentage # Enforces 0-100

config = ServerConfig()
config.port = 70000
```

Impossible states are made impossible to represent. You cannot create a ServerConfig with an invalid port.

Domain-Specific Security and Sanitisation.

Type	Malicious Input	Sanitised Result
`Safe_Str__Username`	`"admin' OR '1'='1"` →	`"admin_OR_1_1"`
SQL Injection Neutralised		
`Safe_Str__Html`	[redacted]	[redacted]
	[redacted]	
`Safe_UInt__Port`	`70000` →	`ValueError` Exception
Strict 0-65535 Bounds Enforced		

Protection at the Source: The Efficiency Impact.



75%

Reduction

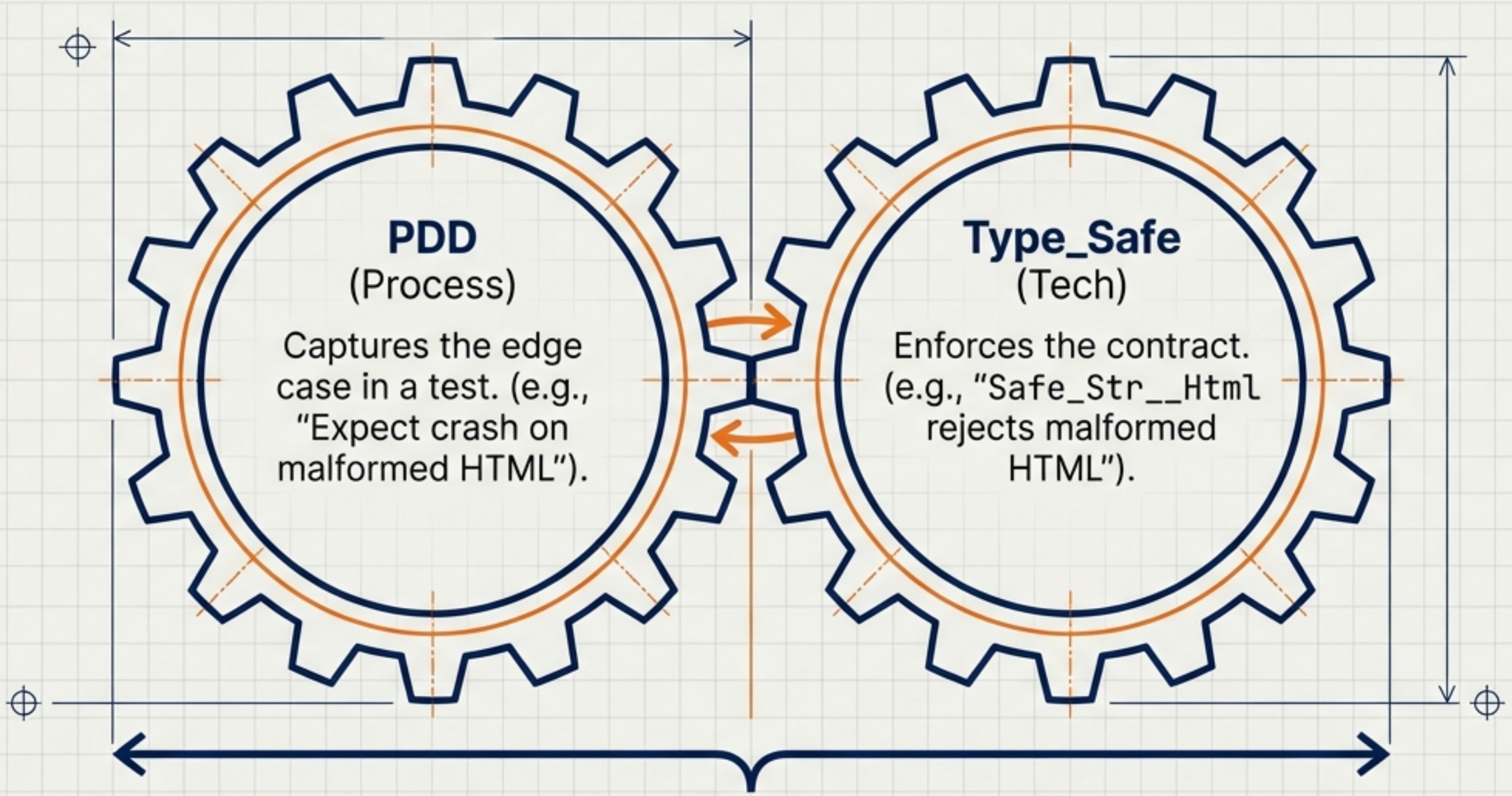
In Debugging Time.
Errors caught at
assignment line.

40%

Reduction

In Testing Burden.
Trivial edge cases
handled by types.

Synergy in Practice: Early Failure, Early Fix.



Result: A system that fails fast on controlled terms. If the type check misses it, the PDD test catches it. If the PDD test is missing, the type check prevents corruption.

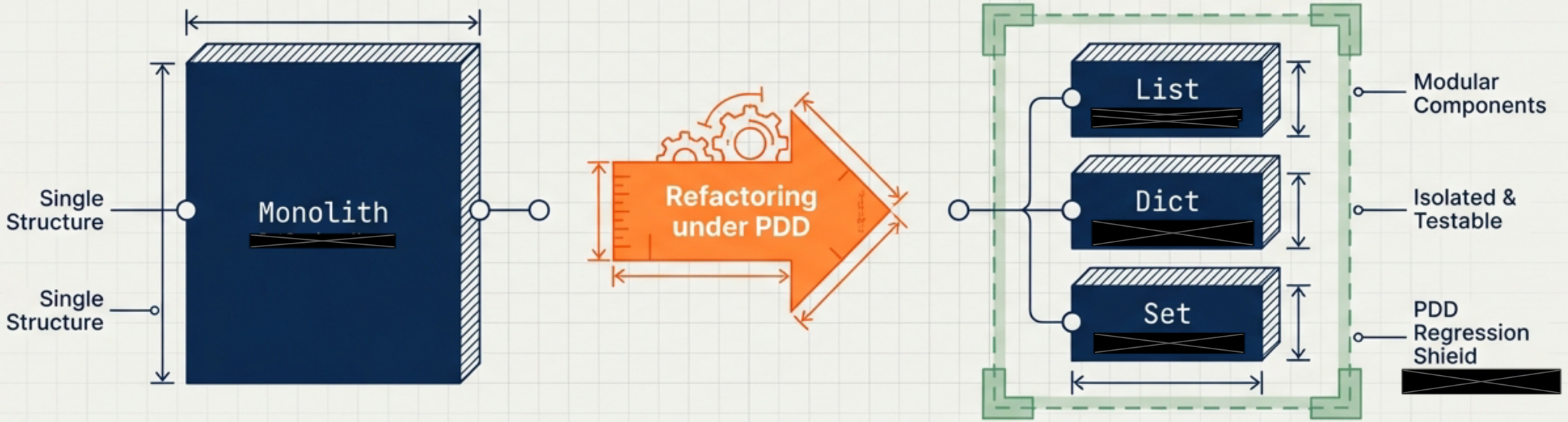
Case Study: Fearless Refactoring

The Challenge

Dinis Cruz refactoring core Type_Safe structures.
Splitting one monolithic class into three.

The Outcome

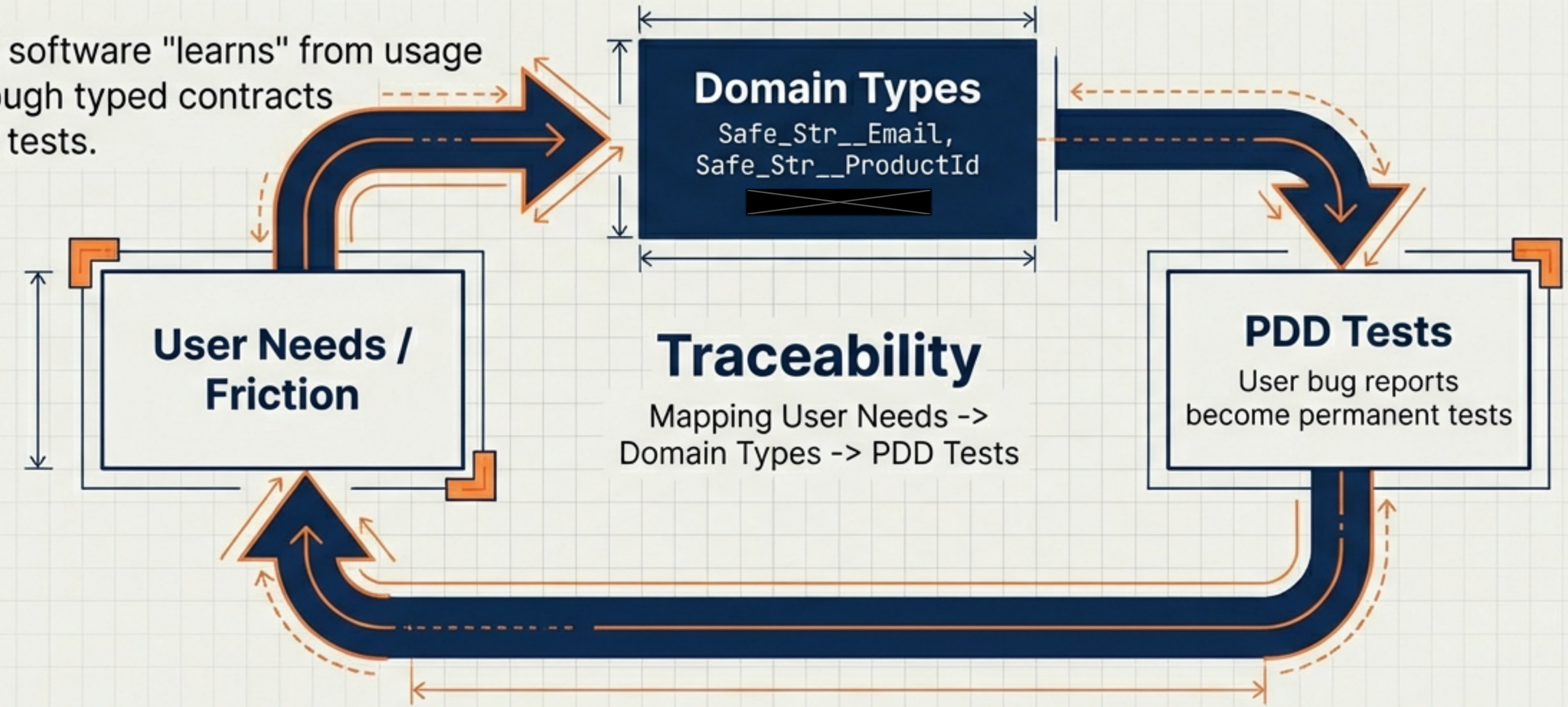
High coverage allowed massive structural changes.
Failing tests acted as a roadmap, not a blocker.



“High test coverage + PDD = The confidence to aggressively evolve architecture.”

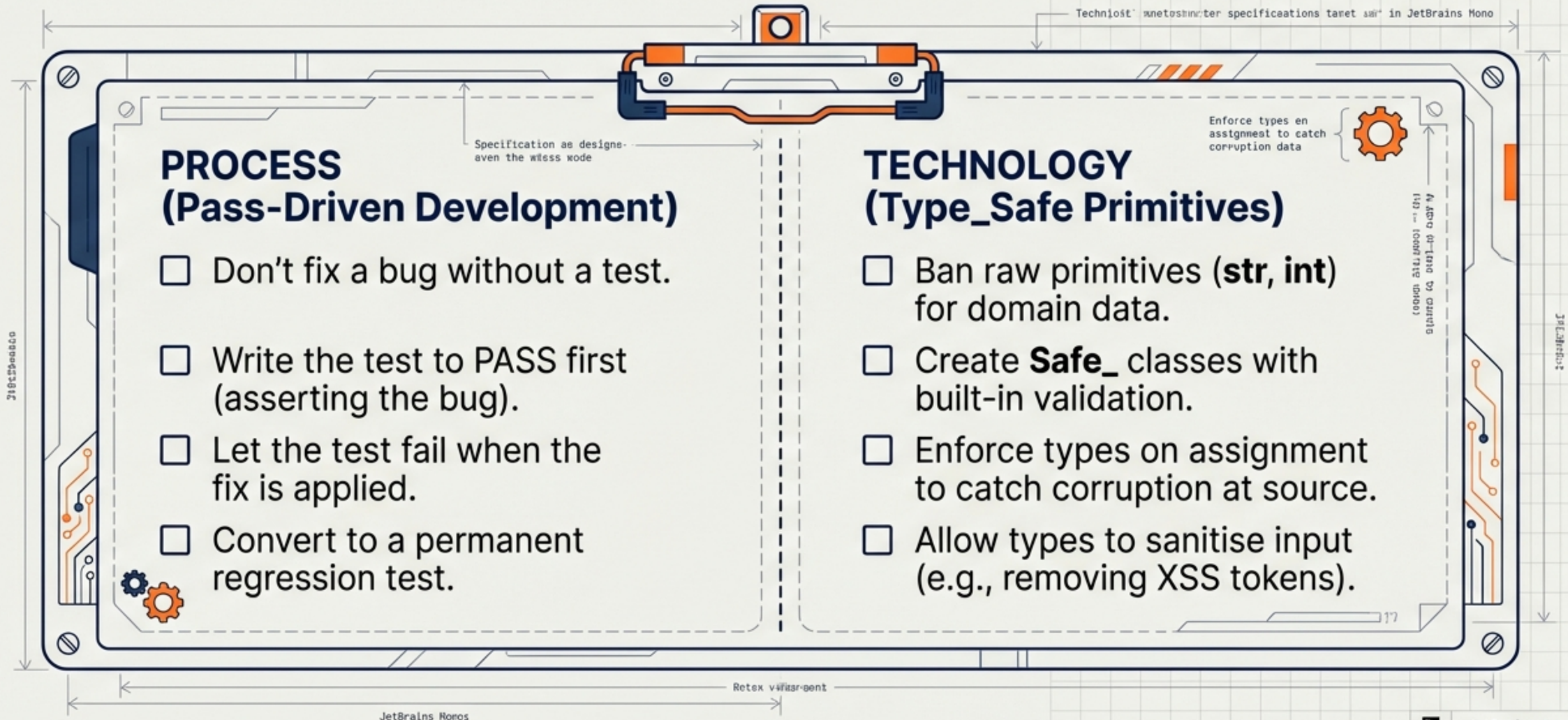
Building Consumer-Aligned Software.

The software "learns" from usage through typed contracts and tests.



Result: Fewer silent data corruptions and a system that behaves predictably.

Implementation Checklist for Resilientt Engineering.



PROCESS (Pass-Driven Development)

- ☐ Don't fix a bug without a test.
- ☐ Write the test to PASS first (asserting the bug).
- ☐ Let the test fail when the fix is applied.
- ☐ Convert to a permanent regression test.

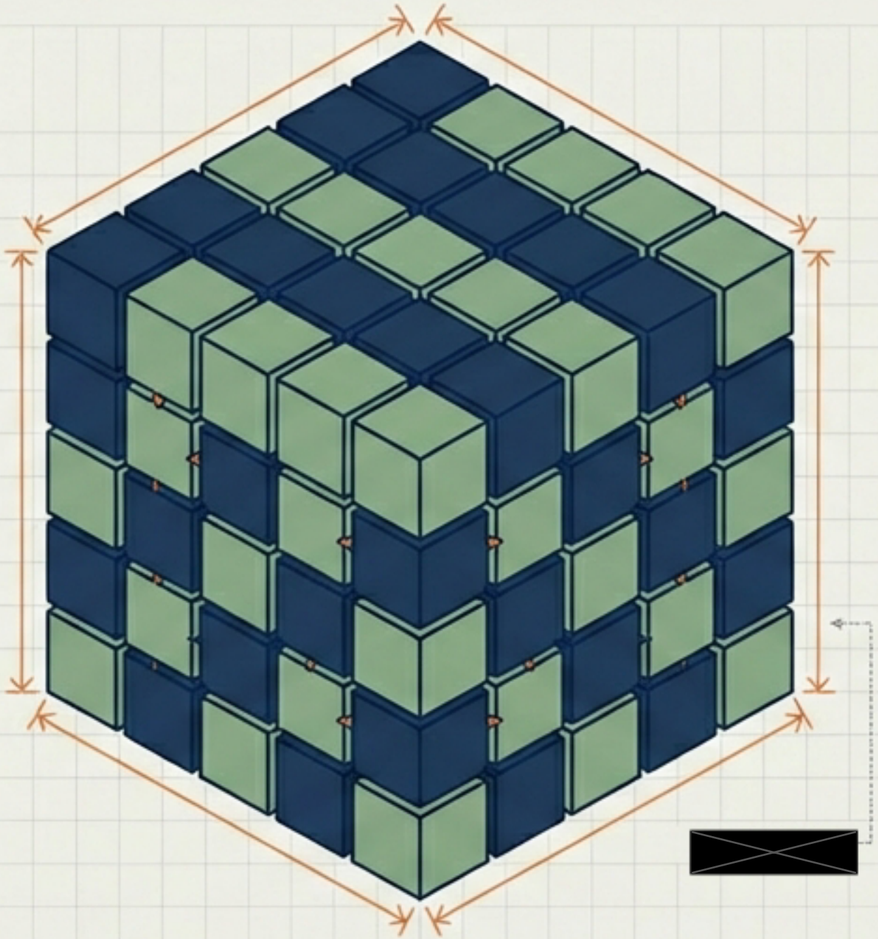
TECHNOLOGY (Type_Safe Primitives)

- ☐ Ban raw primitives (**str**, **int**) for domain data.
- ☐ Create **Safe_** classes with built-in validation.
- ☐ Enforce types on assignment to catch corruption at source.
- ☐ Allow types to sanitise input (e.g., removing XSS tokens).

Conclusion: Quality is Baked In, Not Bolted On.

PDD creates a living documentation of the system's journey and a shield against regression.

PDD creates a living documentation of the system's journey and a shield against regression.



Type_Safe Primitives bring the rigour of static systems to dynamic runtimes.

Type_Safe Primitives bring the rigour of static systems to dynamic runtimes.

Call to Action: Adopt **'No Bug Left Behind'** and a **'Type-Safe First'** Architecture.